Mathrix_AI_Coding_Challenge.docx
DOCX

LiveArmenianTranscriptionEnglishTranslation
FOLDER

this is my task, and this is my system parameters
Python : 3.12.10 64bit
OS : Windows-11-10.0.26300-SP0
CPU : Intel64 Family 6 Model 140 Stepping 1, GenuineIntel | logical cores: 4
RAM : (pip install psutil to see)
GPU : no NVIDIA GPU / driver found -> CPU-only plan
read it

Read the attached task document

I've read the brief. Here is what it asks for and how it fits your machine.

The task: a Python program that captures the microphone and the system audio (the other side of a call) at the same time. It transcribes Armenian speech, translates it to English, and prints `[time] Speaker / HY / EN` lines live. The English line must appear within 5 seconds of the end of a phrase.

Constraints:

- Everything runs locally, with no cloud APIs.
- Mic audio is labelled "Me". System audio is split into Speaker 1, 2, … and each person keeps their label for the whole session.
- Deliverables: a git repo with the code, a README, and your Claude chat exports as `.md` files in `chats/`. The README must cover setup, model choices with alternatives, architecture, and limitations.
- Ignored: GUI, echo handling, saving transcripts, and speaker names.
- Use only Claude Cowork. Export every session in full and unedited, and keep the originals available for the interview.

What your hardware means: you have 4 logical cores, no GPU, and Windows 11. That makes the 5-second limit the hard part.

- **Transcription:** Whisper large is too slow on CPU. Candidates are `faster-whisper` with int8 quantization on a small or medium model, or an Armenian fine-tuned model. Armenian quality varies a lot between models, so we should benchmark them on your machine.

- **Translation:** a local model such as NLLB-200-distilled-600M or MarianMT, run through CTranslate2 for CPU speed. I'm not sure a Marian model for Armenian exists, so I'd check.
- **Speaker identification:** speaker embeddings (e.g. SpeechBrain ECAPA or Resemblim-style) with online clustering. A VAD such as Silero would find the phrase boundaries.
- **Audio capture on Windows:** WASAPI loopback, which needs no virtual device. `soundcard` or `pyaudiowpatch` can capture it.

Open question: should the ASR and translation choice be decided by benchmarking? I'd suggest I scaffold the repo and write a small benchmark script for you to run on your PC, since I can't reach your audio hardware from here.

Which would you like me to do first?

1. Scaffold the repo structure and the audio capture and VAD pipeline.
2. Research and benchmark Armenian ASR and translation models for CPU.
3. Draft the README skeleton and architecture.

Research and benchmark Armenian ASR and translation models for CPU.

Read 6 pages, searched the web, and 6 more steps ▸

Research is mostly done. I've found the Armenian ASR candidates and confirmed an Armenian→English Marian model exists. I'm now building the benchmark scripts for your PC.

Ran 5 commands, updated tasks ▸

Benchmark scripts are written and the FLEURS Armenian test clips are downloaded. I'm waiting for the package install to finish so I can smoke-test them here before handing them over.

Ran 6 commands ▸

Smoke tests are going well. The Armenian speech model (FastConformer int8) runs at about 0.05× real time here, and NLLB translates much better than the Marian model. I'm fixing a small scoring bug, then running the full comparison, including Whisper as a baseline.

Ran 6 commands ▸

I'm still waiting on the Whisper baseline. It's slow on this 2-core sandbox, and I'm treating that as part of the result. The benchmark scripts are packaged. Next I'll copy them to your project folder and write up the recommendation.

Ran 2 commands, updated tasks, saved files to your computer ▸

I benchmarked the Armenian speech models and the translation models, and I recommend the NVIDIA FastConformer speech model with NLLB-600M for translation. I ran everything in a 2-core Linux sandbox, not on your PC. The scripts are in `bench\` in your project folder, and you should re-run them there.

Speech recognition (20–40 FLEURS clips of about 11 seconds each)

Model	Word error rate	Latency per clip	Verdict
FastConformer int8	2.5%	about 0.5 s (0.05× real time)	Use this
Whisper small (multilingual)	95%	about 12 s (3.2× real time)	Unusable for Armenian on CPU

- **FastConformer:** the Vana-Labs int8 ONNX export of NVIDIA's Armenian model has about 115M parameters and is CC-BY-4.0. It outputs punctuation and runs with plain `onnx-asr` and `onnxruntime`, with no GPU or NeMo needed.
- **Whisper:** I didn't run the larger Whisper models. They are heavier than small and would be even slower on 4 CPU cores.
- **Optimistic WER:** FLEURS was part of that model's training data and is clean read speech. NVIDIA reports 9.9% on Common Voice and 12.3% on FLEURS, and real calls will be worse. Re-test on a recording of your own speech.
- **Armenian Whisper fine-tunes:** Chillarmo's large-v3-turbo (15.3% WER, self-reported) is about 0.8B parameters and its card recommends 8 GB of GPU memory. I didn't benchmark it, and I'd expect it to miss the 5-second limit on your CPU.

Translation (Armenian to English)

Model	BLEU, true text	BLEU, from the speech model's output	Latency per sentence
NLLB-200-distilled-600M (int8)	31.4	28.6	about 2 s, up to 3.8 s
Helsinki opus-mt-hy-en	13.9	13.0	about 0.5 s

NLLB is much better. The Marian model makes things up, for example turning "Aristarchus" into "Manfred", so I'd use NLLB. The 2 s average is the main risk to the 5-second limit, and your 4 cores should make it faster. If it's still too slow, translate each sentence as soon as it ends instead of waiting for the whole phrase.

Still to do

- **Speaker identification:** not benchmarked. A speaker-embedding model such as SpeechBrain ECAPA with online clustering is the likely choice, but I haven't tested it.

- **Call audio:** nothing here has been tested on compressed call audio or overlapping speakers.
- **Python version:** the scripts ran on Python 3.13 here. I haven't run them on your Python 3.12.

To run the benchmarks on your PC, follow the setup steps in `bench\README_bench.md`. Send me your numbers and I'll draft the model-choice section of your README.

ok, now lets start from speaker detection, mic will be me and as speakers i will use youtube videos or telegram calls

Searched the web, ran 5 commands ▸

First speaker-identification results are in. On simulated Armenian calls with clean audio, the best embedding models label 3–4 speakers about 97% correctly. I'm now testing with phone-call-style compression, which is closer to your YouTube and Telegram setup.

Ran 2 commands, saved files to your computer ▸

Speaker detection works in my tests. I've put the code in `speaker\` in your project folder. These results come from a 2-core Linux sandbox, so please re-run them on your PC.

How it works

- Your mic needs no detection, so everything from it is labelled "Me".
- For system audio, each spoken phrase is turned into a voice fingerprint (an embedding) by a small local model.
- `OnlineSpeakerTracker` in `speakers.py` compares each fingerprint with the speakers heard so far. If it's close enough to one of them, the phrase gets that label, and otherwise it starts a new "Speaker N". That keeps the same person on the same label for the whole session.
- Phrases under 1 second are too short to judge reliably, so they reuse the closest known speaker without changing it.
- It uses sherpa-onnx, which installs with pip. It needs no PyTorch or GPU and is Windows-friendly.

Test setup: I built simulated calls with 2, 3 or 4 speakers from real Armenian recordings in Common Voice 20, which has speaker IDs. Some runs squeezed the audio through low-bitrate Opus, which is roughly what Telegram uses.

Model	Accuracy, clean audio	Accuracy, Opus-compressed	Time per phrase
NeMo TitaNet-small	97%	97%	about 70 ms
3D-Speaker CAM++	98%	96%	about 80 ms
3D-Speaker ERes2Net	94%	91%	about 165 ms
WeSpeaker ResNet34 (CN-Celeb)	92%	87%	about 120 ms
WeSpeaker ResNet34 (VoxCeleb)	78%	not run	about 120 ms

TitaNet-small and CAM++ are clearly the best, and either is fast enough for the 5-second limit. Cropping phrases to 2 seconds barely changed this (97% for TitaNet, 95% for CAM++). I'd start with TitaNet-small at a similarity threshold of about 0.45.

Caveats

- **Easy test:** each Common Voice speaker recorded on their own microphone and room, which makes them easier to tell apart. In a YouTube video all voices share one audio channel, so expect lower accuracy. Please test with 2–3 real videos.
- **Tuned threshold:** the 0.45 threshold was tuned on this same data. It will probably need adjusting once you try real calls.
- **Overlap:** overlapping speech and very short interjections like "yes" or "uh-huh" are not handled.

Next, I'd build the audio capture on Windows: your mic plus the system audio, with voice-activity detection to cut phrases. Should I start on that, or first fetch a few real multi-speaker YouTube-style test clips to tune the threshold?

how to run what we have now

Preparing Windows run instructions for the project.

Run all of this in PowerShell, from your project folder. It has not been run on Windows yet, so tell me if anything fails.

1. One-time setup

powershell

```
cd C:\Users\User\Desktop\LiveArmenianTranscriptionEnglishTranslation
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install onnx-asr onnxruntime ctranslate2 transformers sentencepiece jiwer sacr
pip install torch --index-url https://download.pytorch.org/whl/cpu
winget install Gyan.FFmpeg
```

- Torch is only needed once, to convert the NLLB and Marian translation models.
- Close and reopen PowerShell after installing FFmpeg, so `ffmpeg` is found. If the venv won't activate, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` first.

2. Speech recognition and translation benchmarks

powershell

```
cd bench
python fetch_data.py                      # downloads about 210 MB of Armenian tes
python bench_asr.py --models fastconformer-int8 --n 40
python bench_mt.py --models opus-mt nllb-600m --source gold --n 40
python bench_mt.py --models nllb-600m --source asr --n 40    # speech model output
```

- The first run downloads the models, which takes a few minutes.
- `bench_mt.py` converts each translation model the first time it uses it, which also takes a few minutes.
- You can skip `whisper-small`, because it was unusable for Armenian in my tests.

3. Speaker identification benchmark

powershell

```
cd ..\speaker
python fetch_speaker_data.py              # four speaker models and a 200 MB datas
python bench_speakers.py --models models\nemo_en_titanet_small.onnx models\3dspeak
python bench_speakers.py --degrade opus --models models\nemo_en_titanet_small.onnx
```

Run `bench_speakers.py` from inside `speaker\`, because it looks for `models\` and `data\` relative to that folder.

What to look for

- **Speech recognition:** the p95 latency per clip should be well under 5 seconds, and the real-time factor should be around 0.1 or lower.

- **Translation:** latency per sentence on your 4 cores. The NLLB-600M model is the one that might struggle with the 5-second limit.
- **Speakers:** the line `best thr ... mean acc`, which gives the accuracy and the best similarity threshold for each model.

This is only benchmarking. There's no live program yet, so nothing listens to your mic or system audio. Send me the numbers when you have them, and then I'll build the live capture part.

Pasted text.txt

TXT

Shared 2 files, used Computer, ran 3 commands

Speech recognition works on your PC. Translation and speaker identification each failed on a separate problem, and I've fixed both in the two files I just sent.

Speech recognition (passed)

- On your 4 cores the Armenian model had a 2.4% word error rate on the test clips.
- The slowest clips (p95) took 0.45 s, about 0.02× real time. Speech recognition will use very little of the 5-second budget.

Translation (failed, now fixed)

- The script ran the `ct2-transformers-converter` command by name, but pip didn't put it on your PATH.
- The new `bench_mt.py` calls the converter through Python instead. I tested that form here, and it converts the Marian model.

Speaker identification (failed, now fixed)

- `nemo_en_titanet_small.onnx` is corrupt. The error "Protobuf parsing failed" means the file didn't download fully, and the fetch script skipped it because a file with that name already existed.
- The new `fetch_speaker_data.py` re-downloads any model file smaller than 10 MB. To be safe, delete `speaker\models\nemo_en_titanet_small.onnx` first.

To redo

1. Replace `bench\bench_mt.py` and `speaker\fetch_speaker_data.py` with the two files I just sent.

2. Run these from the same folders, **without the `# ...` comments** (your command prompt read them as arguments, which caused the "unrecognized arguments" error):

```
cd bench
python bench_mt.py --models opus-mt nllb-600m --source gold --n 40
python bench_mt.py --models nllb-600m --source asr --n 40
cd ..\speaker
del models\nemo_en_titanet_small.onnx
python fetch_speaker_data.py
python bench_speakers.py --models models\nemo_en_titanet_small.onnx
models\3dspeaker_speech_campplus_sv_zh_en_16k-common_advanced.onnx
python bench_speakers.py --degrade opus --models
models\nemo_en_titanet_small.onnx
```

Notes

- **Virtual environment:** `.venv\Scripts\Activate.ps1` only works in PowerShell. You're in Command Prompt, so nothing was activated and the packages went into your user Python. That's fine for now.
- **HuggingFace warnings:** the symlink warnings are harmless.
- **Connection:** I lost my connection to your computer during this turn, so I couldn't write the files into your project folder, which is why I sent them as attachments.

Send me the translation and speaker numbers when they finish.

Download all

Pasted text.txt

TXT

Weighing translation and speech timing against the latency budget ▸

All three parts now pass on your PC, and the main risk to the 5-second limit is translation, not speech recognition or speaker detection.

Part	Choice	Result on your PC
Speech recognition	FastConformer int8	2.4% word error rate, p95 0.45 s
Translation	NLLB-200-600M	p50 1.5–1.7 s, p95 3.0–3.7 s per sentence
Speaker identification	CAM++ (or TitaNet-small)	98–99% accuracy, about 45 ms per phrase

- **Translation quality:** NLLB scores BLEU 28.9 on the speech model's own output, against 14.0 for the Marian model (the 28.9 is on the speech model's output, the 14.0 on the true text). The Marian model invents names (it turned "Aristarchus" into "Manfred"), so I'm dropping it.
- **Speaker identification:** both models do best at a similarity threshold of 0.40, and accuracy stays high from 0.40 to 0.50. I'd use CAM++ at 0.45. Opus compression made no real difference (98% for TitaNet).
- **Quality caveat:** the speaker test used Common Voice, where each person recorded on their own microphone. YouTube and Telegram voices share one audio channel, so the threshold will need re-tuning on real clips.

Latency risk: the 5 seconds are counted from the end of a phrase. In the worst case that is 0.5 s of silence detection, 0.45 s of speech recognition and up to 3.7 s of translation, which adds up to about 4.7 s. These are the ways I plan to cut it:

- **Speaker identification:** run it at the same time as translation instead of before it.
- **Translation:** use greedy decoding, which is faster than the beam search I used in the benchmark.
- **Long phrases:** cut anything over about 6–8 seconds into pieces.
- **Output:** print the Armenian text as soon as it's recognised, and the English a moment later.

Next step: build the live program, in four parts:

1. WASAPI loopback capture of the system audio, plus your mic.
2. Voice-activity detection to cut phrases.
3. A pipeline that runs speech recognition, speaker identification and translation in separate threads.
4. The `[time] Speaker / HY / EN` terminal output, with a flag for choosing devices.

I'll start on that when you say go. I'd also like you to record a short YouTube clip with two speakers and a Telegram call to test with, so I can check the threshold against real audio.

go

Ran 5 commands, updated tasks ▸

The live pipeline works end to end in a simulated call. It labelled all three speakers correctly, and the worst delay from end of speech to printed English was 3.0 s, on my slower 2-core sandbox. I can't test the real microphone and system-audio capture from here, so that part is untested until you run it. I'm now copying the files into your project folder.

Updated tasks, ran a command, saved files to your computer ▸

The live program is written and in your project folder. It runs end to end in a simulated call, but I couldn't test the real microphone and system-audio capture from here, so that is the first thing you need to try.

Test result: I played a 90-second simulated call made from real Armenian recordings: one voice as your mic and two voices as the call.

- **Speaker labels:** all 12 phrases got the right label. The mic voice was "Me" and the two call voices were Speaker 1 and Speaker 2.
- **Delay:** the delay from end of speech to printed English was 1.1–3.0 s. That was on my slower 2-core sandbox, so your 4 cores should be a little faster.
- **Translation quality:** it is usable, with some mistakes. For example, "egg" came out where a bird's egg was meant, and it mistranslated a medical term.
- **Mic voice split in two:** a phrase of yours with a long pause in it was cut into two lines, because a 0.45 s pause counts as the end of a phrase. `--min-silence 0.7` makes it split less, at the cost of a bit more delay.

Run it (from your project folder, with the packages from before already installed):

```
pip install PyAudioWPatch
python setup_models.py
python -m live --list-devices
python -m live --show-latency
```

- `setup_models.py` downloads the models and converts the translation model once. It is slow the first time.
- `--list-devices` shows your microphones and system-audio (loopback) devices with ids. By default the program uses your Windows default microphone and the loopback of your default speakers or headphones.
- If it picks the wrong device, add `--mic ID` or `--loopback ID`.
- For YouTube or a Telegram call, play the audio through your normal output and the program should hear it. If you use headphones, your microphone won't pick up the call, and echo handling is out of scope anyway.

To test without audio devices:

```
python tools/make_sim_call.py --parquet speaker/data/cv_test.parquet --out sim
python -m live --sim sim_mic.wav sim_system.wav --show-latency
```

Known gaps

- **Echo:** the program does nothing about speakers playing into your mic.
- **Cut-off sentences:** phrases longer than 8 seconds are cut in two.
- **Speaker threshold:** the 0.45 setting was tuned on Common Voice, so it may need adjusting on real YouTube or Telegram audio (`--spk-threshold`). Lower it if one person gets split into two speakers, and raise it if two people get merged.
- **README:** I haven't written the README, and your repo still needs the `chats/` exports.

Run it on a real call and tell me what you see, especially any error from the audio devices. If it works, I'll write the README next.

Pasted text.txt

TXT

see i have a problem said Բարև Ձեզ or ինչպես եք, but when my voice not very clear it don't understood well but real person can speak like it, also this is youtube transcript you can compare
Chapter 1: Հյուրընկալություն և նորություններ
0:077 secondsՄենք վերադարձանք և ժամանակն է ներկայացնելու մեր առաջին հյուրերին: Ամառային արձակուրդից հետո վերսկսել են դասերը ոչ միայն հանրակրթական
0:1515 secondsդպրոցներում, այլև մի գեղեցիկ օր հեռուստասերիալում: Եվ այսօր մեր երկրի թերևս ամենահայտնի աշակերտներին ենք
0:2222 secondsհյուրընկալել: Մեր տաղավարում են դերասաններ Էրիկ Թամամյանը և Մարալ Գասպարյանը լույսի լույս
0:3030 secondsուրախ ենք ձեզ երկրորդ անգամ տեսնել առավոտը շանթումի տաղավարում և վստահ եմ, որ մեր հեռուստադիտողը նույնպես շատ ուրախ
0:3636 secondsէ այս առիթով եկեք հասկանանք նախորդ անգամ մեզ հյուր գալուց հետո ինչ է փոխվել ձեր
0:4343 secondsհարաբերություններում ձեր կերպարների հարաբերություններում ընկերական հարաբերություններում, որովհետև այն ժամանակ սերիալը նոր էր սկսվել և դեռ իրար
0:5151 secondsչէիք ճանաչում, շփում չկար այդքան, հասցրե՞լ եք մտերման շատ, շատ ենք մտերմացել բա բավականին այո Ըհը Ու առանց իրար շատ
0:5959 secondsդժվար ա հիմա իրականում, նույնիսկ, որ գնացել էինք

արձակուրդներ, մենք պարբերաբար հանդիպում էինք, որովհետև կարոտում էինք իրար շատ ու մի քանի օր
1:081 minute, 8 secondsառաջ հետաքրքիր բան եղավ նկարման ընթացքում սենց իրար հետ խոսում էինք ու ասեցինք՝ ինչ հետաքրքիր ենք, երբ որ սենց զրուցում ենք ուղղակի [ծիծաղ] իրար հետ
1:151 minute, 15 secondsնկարումից դուրս ու Գուրգենը միանգամից երեխեք կարո՞ղ է այսօր գաք մեր տուն շփվենք իրար հետ ու այդպես հասկացանք, որ մենք նաև ուզում ենք շփվել իրար հետ, հա
1:231 minute, 23 secondsու շատ մտերմացել ենք: Իսկ այս մտերմությունը արդյոք օգնում է նաև սերիալ սերիալի նկարահանման տաղավարում, արդյոք
Chapter 2: Մտերմությունը նկարահանման հրապարակում
1:311 minute, 31 secondsօգնում է նաև սերիալի որակային բարձրացման հարցում և օգնում է և խանգարում, որովհետև հումորները շատացել են մի փոքր, արդեն
1:401 minute, 40 secondsշատ-շատ ենք ծիծաղում ռեկերից առաջ, բայց շատ օգնում է իհարկե նաև ըի իսկ վերջին անգամ կհիշե՞ք ինչի վրա եք այդպես լիացող
1:481 minute, 48 secondsծիծաղել, որ նույնիսկ գուցե չգիտեմ նկատողություն եք ստացել, որ մի քիչ հանգիստ թողեք նկարենք: [ծիծաղ]
1:551 minute, 55 secondsՊատկվի ես երևի բոլոր օրերում էլ այդպիսի ինչ-որ կադր լինում է, բայց վերջինը կհիշեմ,
2:022 minutes, 2 secondsտեսարան ենք անցնում, ես պիտի լուրջ-լուրջ սենց խոսամ, խոսամ, խոսամ, Մարալի կերպարը ինքը ոնց որ իմ ուզած պատասխանը ինձ տալիս
2:112 minutes, 11 secondsա, ես պիտի ժպտամ ու ժպիտ թաքցնեմ էլի ու շարունակենք մեր զրույցը: Սենց Էս դեմքից մի հատ սենց ժպտում եմ ու էլ հասկանում
2:192 minutes, 19 secondsեմ, որ էլ չեմ կարողանում էդ ժպիտը թաքցնում [ծիծաղ] ու սենց մի հինգ րոպե նայում եմ Մարալի դեմքին: [մաքրում է կոկորդը] Լավ, իսկ այդպես մնաց, թե՞ հետո նորից նոր
2:282 minutes, 28 secondsչէ դե ես քանի որ պրոֆեսիոնալ եմ Հասմիկ ջան իմ գործում այո [ծիծաղ] ասում մենք պրոֆեսիոնալների ենք հյուրընկալել
2:372 minutes, 37 secondsայո ՉԷ կատակը մի կողմ մի 10 րոպե ծիծաղում եմ և անցնում ենք առաջ Բավականին ժամանակ է ինձ սերիալը արդեն
Chapter 3: Սերիալի ուսուցողական դերը և բուլինգը
2:442 minutes, 44 secondsթեթերում է բավականին ակտիվություն ունի չեմ ուզում այլևս խոսել ռեյտինգների մասին քանի որ նրանք բավականին բարձր են և ի սկզբանե երբ խոսում էինք եթե հիշում եք
2:522 minutes, 52 secondsսերիալն ուներ ուսուցողական բնույթ հա այ տեսնում եք մարդկանց ովքեր այսքան ժամանակ դիտելով սերիալը որոշակի փոփոխություններ and here i also have problem when someone speak after other it not detect as new speaker if it is in same 22s

Show more

Ran 10 commands, updated tasks, 1 failed ▸

Progress on your two problems: speaker changes inside one phrase now get split, and I fixed the 20-second latency reading. The latter was a measurement artifact, because Windows system-audio capture sends no data during silence. I'm testing the speaker split on a harder simulated call.

Ran 11 commands, updated tasks, saved files to your computer ▸

The fixes are in your project folder. I'm writing up what I changed and what I found along the way.

I fixed the 20-second latency reading and the missing speaker changes, and made the speech cutting more accurate. Your own unclear voice is only partly fixed. I tested everything on simulated calls and the real Windows audio capture is still untested. Your folder now has the updated project, with a few extra files I found along the way (see below).

The "20 s latency" and wrong timestamps weren't real. Windows system-audio capture sends no data while nothing is playing, so the program's clock fell behind real time by however long your YouTube video hadn't started. It also meant the last phrase before a pause could sit unprocessed until sound resumed. The program now fills those silent gaps and reads the real clock. In my "silent-gap" simulation the timing and labels now match the normal run. Real latency in your log is probably about 2–4 s.

Back-to-back speakers. When one person answers with no pause, the program used to treat both as one phrase and give one label. Phrases from the call audio are now split at speaker changes, and each part gets its own label. The test used 10 turns with no pause and clips trimmed tight.

- With splitting off, those turns collapsed into 4 lines, all labelled Speaker 1.
- With splitting on, the labels followed the real speakers, with a few turns cut into extra lines.
- There are two ways to split, `--splitter pyannote` (default) and `--splitter embed`. Both gave correct speakers on that test. `--no-split` turns splitting off.

Your voice ("Բարև Ձեզ" heard as "Պարև ձեզ"). Two causes showed up in my tests:

- **Clipped starts:** the program cut each phrase exactly at the speech edge, which clips soft consonants like Բ. Adding 0.25 s of audio before and 0.15 s after each phrase cut the word error rate from 9.0% to 6.7% on Common Voice.
- **Noise:** with background noise (10 dB below your voice) the error rate was 21.7% against 6.7% clean. Quiet voice and volume normalisation made no difference. A close microphone or headset in a quiet room is the biggest improvement left.
- **Greeting lookup:** `live/lexicon.py` translates short greetings and one-word answers directly, because the translation model makes up English for them. It also tolerates Պարև/Բարև-type confusions.
- **Echo:** the lone "Հ:" and "Հերես:" lines at 2:32–2:35 were probably your speakers playing into your mic. Use headphones when testing.

Something you should know. When I checked the project folder, it already held files I had not written in this conversation: `lexicon.py`, `turns.py` (the pyannote splitter), a changed `translate.py`, `--dump` and `--sim-loopback` flags, and a segmentation model download in `setup_models.py`. They look like another session's work on the same problems. I kept them and wired them in. My earlier rewrite had left that splitter loaded but unused, and I reconnected it. I also may have overwritten parts of the other session's `pipeline.py`, `vad.py` and `audio.py`, so please check that nothing you wanted is missing.

Still open

- **Real speaker threshold:** the 0.45 speaker threshold and the splitting still need checking on real YouTube audio. Run it with `--dump dumps` and send me a line that looks wrong. The dump saves each recognised phrase as a sound file and text.
- **Delay under non-stop speech:** on my 2-core sandbox, continuous talk with no pauses reached 5–7 s. Your 4 cores should be faster, but please test with `--show-latency` on a fast interview.
- **Setup:** run `python setup_models.py` again to download the new segmentation model.

[00:00:02] Me (latency 1.0s)
HY: Բարև ապուշ:
EN: I'm going to be a little late.

[00:00:06] Me (latency 1.0s)
HY: Բարև, ապուշ:
EN: Hello, my name is Ash.

[00:00:12] Me (latency 0.8s)
HY: Մտքերի տ.
EN: The mind is not alone.

Show more

[00:00:18] Me (latency 0.9s)
HY: ֊ Բարև, հիմար:
EN: - Hey, you fool.

[00:00:26] Me (latency 0.8s)
HY: ֊ Ո՞ւց ես:
EN: - Who are you?

[00:00:33] Me (latency 0.8s)
HY: Ո՞ւց քարտեզն ու:
EN: What map?

[00:00:37] Me (latency 0.8s)
HY: Ո՞ւց ես:
EN: Who are you?

Ran 5 commands, saved files to your computer ▸

Most of what went wrong in your last run was the translation model. I changed three files in `live\` (`asr.py`, `lexicon.py`, `translate.py`) to fix it. I tested the new translation on your

exact lines and on a simulated call, but not with your microphone.

What your log showed

- **Speech recognition:** it did well. It heard "Բարև", "Ո՞նց ես" and "Բարև, հիմար" correctly.
- **Translation:** it failed on short phrases. NLLB invents sentences when given only two or three words, so "Բարև ապուշ" came out as "I'm going to be a little late" and "Ո՞նց ես" as "Who are you?".
- **Dialogue dash:** the speech model often starts a line with a dash (`-`), which turned "How are you?" into "- Who are you?".

What I changed

- **Dash stripping:** leading dashes and quote marks are now removed before translation and display.
- **Short phrases:** utterances of up to four words are translated inside a carrier sentence, as if someone were quoting them ("He said, '…'"). With this, "Ո՞նց ես" gives "How are you?", "Ո՞նց քարտեզն ու" gives "Where's the map?", and "Բարև, հիմար" gives "Hello, you idiot".
- **Greetings:** the greeting list now covers colloquial ones: "Բարև ջան" gives "Hi, dear" and "Բարև ապեր" gives "Hello, bro". A trailing "ջան" is handled separately, because the model read it as the name John.
- **Longer sentences:** unchanged, and the simulated call still translates correctly.

What's still imperfect

- **"ապուշ":** it still translates as "buddy", not "idiot". The model softens it. If you want it exact, I can add it to the word list.
- **Mid-word cut-offs:** "Մտքերի տ." was you being cut off mid-word, and it now shows as "My mind". Raise `--min-silence 0.7` if your short pauses are cutting phrases.
- **Speech errors:** if the recognised Armenian is wrong (for example you said "ապեր" and it wrote "ապուշ"), translation can't fix it. Run with `--dump dumps` and I can check the saved audio.

Your next test should use the system audio: play a YouTube interview and send me the output, with `--show-latency`, if anything looks off.

[00:00:03] Me (latency 0.9s)
HY: ˌ Բարև, ինչպե՞ս ես:
EN: - Hi, how are you?

[00:00:08] Me (latency 0.8s)
HY: ˌ Ո՞նց ես:
EN: - Who are you?

[00:00:11] Me (latency 0.8s)
HY: ՝ Ո՞նց ես:
EN: - Who are you?

[00:00:15] Me (latency 0.8s)
HY: ՝ Ո՞նց ես:
EN: - Who are you?

[00:00:17] Me (latency 0.9s)
HY: Բարևը ապոր:
EN: Hello, you are welcome.

[00:00:21] Me (latency 0.8s)
HY: Ապուր...
EN: I'm not sure.

[00:00:24] Me (latency 0.9s)
HY: Ապուշ...
EN: I'm not sure.

Show more

Listed files on your computer, ran 2 commands, and 2 more steps ▸

That last test ran the old code, and the mistake was mine. I committed the three changed files in the same step as copying them into the staging folder. The commit picked up the previous versions, so your folder still had the old `asr.py`, `lexicon.py` and `translate.py`. The output you pasted fits that: the dash, "Who are you?" and "Hello, you are welcome" are all the old behaviour.

I've re-sent the files and checked their sizes in your project folder. All three now match my tested copies, as do the other files in `live\`. Please re-run `python -m live`.

What to expect on your lines

Your line	New output
`- Ո՞նց ես:`	"How are you?" (no dash)
`Բարև, ինչպե՞ս ես:`	"Hello, how are you?"
Բարև ապուշ	"Hello, buddy."

Not fixed by this

- `Բարևը ապոր:`, `Ապուր...` and `Ապուշ...` are recognition errors, where the speech model wrote different words from what you said. Translation can't fix those.
- **Why:** these are short, soft, one-word utterances, and the model is weakest there.
- **Check:** run with `python -m live --dump dumps` and send me one of the saved sound files and its text.

ok, now write readme

Created a file, ran a command, and 2 more steps ▸

I wrote `README.md` in your project folder and checked that it arrived with the right size. It covers everything the brief asks for, using only numbers we measured:

- **Setup and run:** install steps, all the options, and the Windows audio setup, including that headphones are needed because echo is out of scope.
- **How it fits together:** a short diagram, a table of what each file does, and the problems we found and fixed while testing.
- **Models and alternatives:** a table for each of speech recognition, translation and speaker identification, with the reason each was chosen.
- **Results:** what was measured and what wasn't.
- **Limitations and next steps:** what I would improve with more time.

Before you submit

- **`chats/` folder.** You need to export every Claude Cowork session for this project, unedited, as separate `.md` files. This includes any other sessions, such as the one that added the pyannote splitter and the greeting list. I can't export them for you.
- **README claims.** The README says both "Windows only" and that macOS is not supported. If you want to say more about that, check it.
- **Numbers I couldn't verify on your PC.** The rows for the last three speaker models and the Opus figure for CAM++ come from my 2-core sandbox, and the README labels them that way. Re-run `bench_speakers.py` if you want everything from your machine.
- **Real-call test.** The README says the 5-second limit has not been checked for continuous system-audio speech on your PC after the last fixes. Run a YouTube interview with `--show-latency`, and if the numbers are good, I'll update that line.
- **Git.** I haven't initialised a repository or committed anything. `.gitignore` already excludes models, `.wav` files, datasets and `.venv`.

yes latenncy is ok

Ran a command, saved files to your computer ▸

Good. I updated the README to say that you tested the latency on a live YouTube interview and found it acceptable. I didn't put in a number, because I don't have your figures. If you paste a few `--show-latency` lines, I'll add the measured values.

Before you submit:

1. Put the exported chat sessions in `chats/`. Export every session, unedited, with one `.md` file per session.
2. Run `git init` and commit. `.gitignore` already keeps out the models, `.wav` files, datasets and `.venv`.
3. Keep your original chat history open in Cowork, because they will look at the originals in the interview.

Do a dry run on the laptop you'll bring to the interview. Cover the Telegram call as well as YouTube, and remember to wear headphones.